

HN Breakout: Early Prediction of Hacker News Front Page Posts

MLOps HS26 Project Proposal

David Rivera Allegro

Repository: github.com/AbsturzWindows/hn-breakout (public)

1 October 2026

1 Problem statement

Every day hundreds of new stories are posted on Hacker News (HN), but only a few reach the front page. HN Breakout watches each story for its first 30 minutes and predicts whether it will reach ≥ 100 points within 24 hours. I use 100 points as a measurable way to approximate front page success, since HN does not keep a history of front page stories. To check this, the poller also records the current front page (the top 30 of topstories) every 5 minutes, so I can measure how often 100-point stories actually appeared on it. This is a binary classification task covering every story in the HN newstories feed, including Show HN and Ask HN, from early October 2026. Job postings and deleted stories are skipped.

The system is useful to authors who want to know early whether their post is taking off, and to readers who want to discover likely top stories early. The baseline is simple momentum: stories are ranked by their points after 30 minutes, with ≥ 10 points treated as a hit for binary classification; the threshold is tuned on the training data. The project is successful if, on the final two held-out weeks, the model achieves a PR-AUC at least 0.10 above the baseline and higher precision among its daily top 10 alerts. Because two weeks contain relatively few hits, I report bootstrap confidence intervals for both metrics.

2 Originality & motivation

Why this problem. Hacker News is one of the best known news sites in tech: founders, developers and researchers share and discuss links there every day. In my opinion, its format makes it a good mirror of what people in tech are thinking, both positive and negative, and of what is currently trending. Right now AI dominates the HN front page, but if we had looked at it a few years ago, it probably would have been crypto. Unlike Instagram or Reddit, where everyone has their own front page and a personalized algorithm, HN has one front page for everyone, so it shows the trends of the tech “swarm” as a whole. So predicting which stories make it to the front page shows, in real time, what the tech community is paying attention to.

Why it fits MLOps. Beyond personal interest, the problem is a good fit for MLOps. HN does not keep a history of how a story’s score develops, so if my system does not save the score in time, that information is gone forever. That makes a reliable feature pipeline the most important part of the project, and an interesting challenge I have to get right.

Checked against past projects. I checked both the HSLU projects on mlops-lab.ch and the KTH ID2223 2026 showcase, and did not find any project that predicts the popularity of social news. The closest are “Predict GitHub Trending Repos” (KTH 2026), which works on a different platform on a daily basis, and an older KTH 2022/23 project that predicted Reddit likes. What makes this project different is tracking early momentum: instead of using static post attributes, it learns from how a story gains votes in its first 30 minutes, snapshotted every 5 minutes by a custom poller.

3 Data source & features

Where the data comes from. Everything comes from the official Hacker News API (hacker-news.firebaseio.com/v0). The newstories endpoint gives me the latest ~ 500 story IDs, and `item/<id>` returns the title, URL, author, submission time, score and comment count of each story. The API is public and does not need a key, so there is no scraping involved and no terms-of-service problem. As backfill and fallback I use the Algolia HN Search API, which has past stories with their final points. I use it for domain statistics and to get a feeling for how often stories actually become hits.

How it updates and grows. New stories come in every few minutes. Every 5 minutes the feature pipeline takes a snapshot of each story less than 35 minutes old and saves it with its exact age. Every run is logged, failed API calls are retried, and stories with too many missing snapshots are flagged and left out of training instead of being filled with guesses. After 24 hours it checks the story one last time to get the label. The thing is, HN does not store how a score develops over time, so my training history only starts when the poller starts. With hundreds of stories per day, a few weeks already give me thousands of labelled examples, and the dataset keeps growing linearly. Since polling starts in early October, there will be about eight weeks of data by Milestone three, enough for six weeks of training and two weeks of testing. Before that, I can already test the whole pipeline on the first few days of data. I will report the actual volume in Milestone two.

The label. A story counts as a hit if it reaches 100 or more points within 24 hours of being posted. I get this number by asking the API again one day later. Since the label only exists a day after all the features are collected, none of the features can already know it.

Handling rare hits. I expect only a few percent of stories to be hits (I will estimate the daily story count and hit rate from the Algolia history before polling starts, which also gives the expected number of hits in the test weeks). That’s a problem, because a model that always says “no hit” would look very accurate while being useless. So I train LightGBM with class weights and evaluate it with metrics that focus on how well it finds the rare hits (PR-AUC, precision/recall, and how many of the daily top 10 alerts are real hits) instead of accuracy. If the hit rate estimated from the Algolia history is below 2 %, I switch the threshold to 50 points. This rule is fixed now and applied before any model is trained, so the label is never tuned to model results.

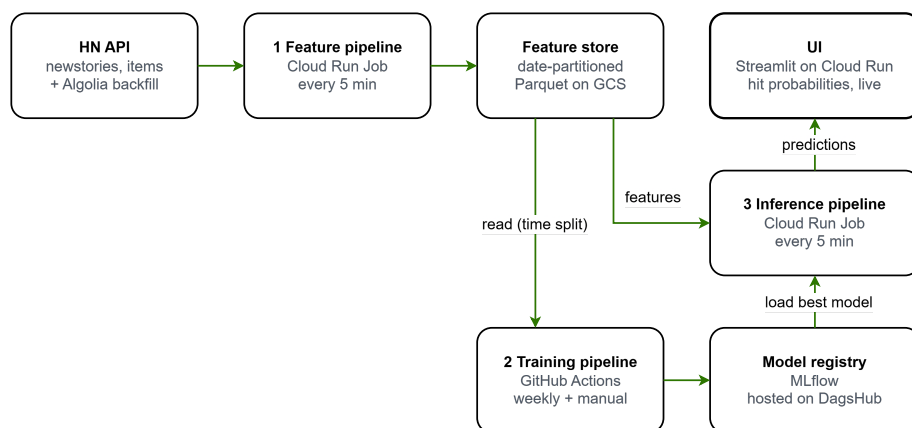
Features. Every feature describes the story when it is at most 30 minutes old:

- **Momentum:** points at 10, 20 and 30 min (from the last snapshot at or before that age), points gained between 20 and 30 min, comments at 30 min, comments per point
- **Timing:** hour of day (UTC), weekday, number of other stories submitted during the story’s first 30 minutes
- **Type and source:** Show HN flag, Ask HN flag (both off for a normal story), has a URL, domain (top 50 domains, the rest grouped as “other”), domain hit rate
- **Title:** length in characters, contains a number, contains a question mark
- **Submitter:** account age and karma (user’s total reputation, similar to Reddit)

Avoiding leakage. To prevent leakage, I split the data by time: older weeks for training, the two most recent weeks for testing. Hyperparameters are tuned only on the training weeks with a time-based validation split, and the test weeks are evaluated once, at the end. Stories younger than 24 hours are left out, because their label isn’t known yet. I also watch out for three specific traps:

- **Late snapshots:** only snapshots taken before a story is 30 minutes old are used as features. Every snapshot stores the story’s age, so this is easy to check.
- **Domain hit rate and karma:** the domain hit rate only counts stories whose result was already known at prediction time, and the submitter’s karma is saved at the 30-minute snapshot, not later.
- **Reposted stories:** HN sometimes gives older stories a second chance on the new page (the “second-chance pool”). I flag these and leave them out of training, because their age doesn’t match when I first saw them.

4 System design



The system follows the FTI pattern: three independent pipelines that only talk to each other through the feature store and the model registry. The feature pipeline writes the snapshots, the training pipeline reads them with an out-of-time split and registers the best model, and the inference pipeline uses that model to score every story the moment it turns 30 minutes old. The feature code is one shared Python module, so training and inference always compute features the same way. The table below shows my stack and why I chose each part.

Component	Choice	Why
Feature pipeline	Python in Docker, Cloud Run Job, started every 5 min by Cloud Scheduler	Snapshots must be on time, and GitHub Actions cron is often delayed by 10+ min
Feature store	Date-partitioned Parquet files on Google Cloud Storage	Simple and cheap, allowed by the course, no extra service to run
Training	GitHub Actions workflow, weekly + manual trigger	Runs once a week, so small start delays don’t matter, same place as the code and tests
Tracking & registry	MLflow, hosted on DagsHub	Compares training runs and stores model versions, free hosted server, nothing to run myself
Model	LightGBM with class weights, compared to the momentum baseline	Strong and fast on tabular data, handles the rare hits well
Inference & UI	Cloud Run Job every 5 min (2 min after the feature job), Streamlit app on Cloud Run	Scores stories as they turn 30 min old, shows a live list with hit probabilities
Quality	pytest + GitHub Actions CI, pinned dependencies (uv.lock)	Required by the course, catches broken feature code early

Optional stretch goals. I will only start these once the core FTI system runs end to end, in this order: drift monitoring of the hit rate and feature distributions with Evidently, Terraform to recreate the GCP resources from code, title embeddings from a small sentence-transformer, and Hopsworks as a managed feature store.